

SkillProx: 通过近端文本梯度下降实现自进化智能体技能

Mingxuan Zheng^{1*}, Yujin Zhou^{1*}, Chuxue Cao^{1*}, Boqin Yin¹,
Yuyao Zhang¹, Jiapeng Sun¹, Shuaishuai Gong², Sirui Han^{1†}, Yike Guo^{1†}

¹Hong Kong University of Science and Technology

²Macau University

mzhengan@connect.ust.hk

摘要

大语言模型智能体越来越多地通过将程序性知识积累为技能来适应重复性任务。这些技能是轻量级、可复用的文本工件，无需更新权重即可加载到智能体的上下文中。近期方法通过迭代任务执行、失败诊断和轨迹引导的文本空间更新来精炼技能。然而，现有框架缺乏显式的诊断-结果反馈，并将删除视为一种通用编辑操作，而非整合累积知识的专用机制。我们提出 SkillProx，一个受近端梯度启发的正反向框架，将闭环诊断演化与效用感知的近端精炼相结合。受平衡任务损失与技能复杂度的复合目标驱动，正向阶段在同一任务批次上重新执行诊断驱动的编辑，回滚退化，并将测量结果反馈到后续诊断中。反向阶段将生成的技能分解为可审计的知识单元，使用冻结的留一法效用审计估计其贡献，并应用验证门控的整合、降级或移除。在多个骨干大语言模型上的分布内和分布外基准实验表明，SkillProx 相比最强的基于梯度的基线平均准确率提高了 3.0 个百分点。组件消融实验证明了闭环诊断与近端精炼的互补效果。¹

1 引言

大语言模型 (LLM) 智能体已展现出通过推理、工具使用与与外部环境交互来解决复杂任务的强大能力 (Yao 等, 2023; Feng 等, 2025; Luo 等, 2025; Team 等, 2026b,a)。为了复用从先前经验中获得的程序性知识，近期智能体越来越多地将任务解决流程、工具使用策略 (Yuan 等, 2024; Lu 等, 2025; Zhou 等, 2026b; Bai 等, 2026) 以及领域特定启发式方法外化为持久且可编辑的技能 (Xu 和 Yan, 2026a; Jiang 等, 2026)。以往研究主要关注技能综合，即从演示、成功轨迹或外部文档中构建可复用技能 (Chen 等, 2023)。然而，SkillsBench (Li 等, 2026) 等基准测试表明，一次性综合¹ 代码将在

<https://github.com/Steven011018/SkillProx> 发布。*These 作者贡献相同。†通讯作者。

在不同任务上可能带来不均衡甚至负面的收益，因为一次性构建的技能无法预见其后续将遇到的多样化情境。这一局限性促使从静态技能构建转向技能演化。代表性方法如 SkillGrad (Wang 等人, 2026a) 将任务失败视为文本梯度，在新任务到来时诊断失败，并直接将推理经验作为补丁提交，从而将静态技能工件调优为程序性记忆的演化形式。尽管取得了这些进展，现有的技能演化方法仍面临两个关键局限。(一) 未经验证的前向更新。大语言模型生成的诊断通常被视为有效的更新方向，而未验证所产生技能编辑的实际效果。补丁一旦生成，通常被直接提交，其有效性既未通过重新执行来衡量，也未作为反馈提供给后续诊断。(二) 不受约束的技能增长。迭代补丁不断扩展技能，却缺乏对累积知识进行重新评估的明确机制。生成的工件可能包含重复指令、冲突启发式，或被错误泛化为可复用规则的任务特定解决方案，其中一些可能干扰有用知识，而不仅仅是增加文本复杂度。这引出一个核心问题：如何将结果验证的前向诊断与结构感知的后向精炼相结合，使技能的任务能力与其累积结构协同演化？为探索这一问题，我们对开环、增长导向的技能演化进行了分析，得出两条可操作的设计原则。在前向方面，我们发现更新不能仅凭其诊断文本判断，但在同一任务批次上重新执行更新后的技能可直接揭示更新是有益还是有害，这表明实际结果应决定更新是否被保留。在后向方面，我们发现累积知识可在单元层面进行审计，移除负效用内容可将准确率从 46% 提升至 54%，这表明技能健康应通过选择性整合而非持续增长来维护。基于这些原则，我们提出 **SkillProx**，一个耦合的前向-后向框架，实现诊断-近端协同演化。为解决未经验证的前向更新，其前向组件在同一任务批次上重新执行每个候选技能，并仅在更新

2026b) 将技能包视为一个结构化参数，通过轨迹级损失证据、文本梯度和动量进行更新。然而，这些方法仍然是开环且以增长为导向的：诊断被接受为有效的更新方向，而未验证其实际效果，因此其结果从未反馈给后续诊断，删除被视为众多通用编辑中的一种，而非专门的收缩机制，导致冗余、冲突或实例特定的内容不断积累，干扰有用知识。SkillProx 通过在前向步骤中闭合诊断-验证回路，并引入一个显式优化复杂度项的效用感知近端步骤，弥补了这两个不足。

3 预备知识与动机 3.1 问题表述 设 X 表示一个完整的结构化技能工件。一个技能由一个主指令文件 `SKILL.md` 和一个可选的 `references/` 目录组成，该目录包含辅助指令、示例或资源。设 $\mathcal{X}$ 表示结构有效技能工件的可行空间。对于任意任务集 D ，设 $H_D(X)$ 和 $C_D(X)$ 分别表示使用技能 X 获得的硬准确率和平均单元准确率。设 $G(X)$ 表示 X 的文本复杂度，实现为 `SKILL.md` 及所有活动引用文件中的字符总数。由于技能是结构化的，它可以分解为知识单元的集合

$$\mathcal{Q}(X) = \{q_1, q_2, \dots, q_m\}, \quad (1)$$

其中每个单元可以对应一个指令部分或一个引用的资源。SkillProx 使用的具体单元分解在第 4.3 节中介绍。从概念上讲，任务损失和文本复杂度定义了以下复合优化问题：

$$\min_{X \in \mathcal{X}} J_\lambda(X) := L_{\mathcal{T}}(X) + \lambda G(X), \quad \lambda \geq 0, \quad (2)$$

其中 $L_{\mathcal{T}}(X)$ 表示 X 在未知任务分布 $\mathcal{T}$ 上的期望损失， λ 表示任务性能与文本复杂度之间的概念性权衡。方程 (2) 刻画了技能演化中涉及的两个目标，而非我们实现中直接优化的目标。文本技能是离散且不可微的，我们的方法不将 λ 作为显式输入。该表述为考察面向性能的技能更新和面向复杂度的知识整合提供了一个共同基础，我们在接下来的两个小节中通过实证加以说明。

3.2 动机

从已实现的更新效果中学习。 在十次 Qwen3.6-27B 训练运行中，开环技能演化平均达到 OJ 困难准确率 50.30 ± 2.50 ，而反馈控制变体达到 51.40 ± 1.51 。对代表性训练轨迹的检查显示，有些更新仅从诊断来看似乎合理，但纳入技能后却降低了任务性能。这一差异表明，文本更新的质量不能仅凭其语义合理性可靠判断，而需要通过其实际执行结果来评估。

仅凭合理性本身无法判断，而需要通过其实际执行结果来评估。在同一任务批次上重新评估更新后的技能，可以直接证明更新是提升、保持还是降低了性能。这一证据既可用于决定是否保留该更新，也可后续诊断提供关于先前有效或无效方向的信息。这些观察促使形成一个诊断、评估和结果反馈的闭环过程，其中技能及其诊断过程根据所提更新的实际效果共同演化。积累知识的整合。代表性演化技能表现出重复的程序性指令和以可复用规则呈现的任务特定解决方案。因此，额外的技能内容并不一定转化为额外的任务解决能力。留一法评估进一步识别出具有负效用的积累知识单元，即移除它们可提升验证性能。在一个代表性案例中，效用感知整合将技能规模缩减了 3.12%，同时将 OJ 困难准确率从 46% 提升至 54%。这些观察促使形成一个反向过程，在单元层面重新评估积累知识，并选择性地移除或整合有害内容，同时保留对任务性能有贡献的知识。上述描述的具体案例见附录 A。

3.3 近端梯度下降

前述观察促使两种互补操作：正向过程评估并改进新提出的知识，反向过程重新评估并整合积累的知识 (Tanabe, Fukuda, and Yamashita 2019)。近端梯度下降为关联这些操作提供了一个自然的框架。对于连续复合目标

$$\min_x f(x) + \lambda g(x), \quad (3)$$

近端梯度下降首先对可微目标执行前向梯度步 f ，然后应用与潜在非光滑正则化项相关的后向近端步 g ：

$$\begin{aligned} v_k &= x_k - \eta \nabla f(x_k), \\ x_{k+1} &= \text{prox}_{\eta \lambda g}(v_k), \\ \text{prox}_{\eta \lambda g}(v) &= \arg \min_x \left\{ \lambda g(x) + \frac{\|x - v\|_2^2}{2\eta} \right\}, \end{aligned} \quad (4)$$

其中 $\eta > 0$ 表示前向步长， $\lambda \geq 0$ 控制正则化强度。前向步主要降低任务目标，而近端步控制所得解的结构或复杂度。技能文本是离散且不可微的，因此不直接支持数值梯度或标准近端算子。我们的实现也未显式优化方程 (2)，或采用 η 和 λ 作为输入。我们仅保留前向-后向的职责划分：诊断演化作为不精确的、面向性能的前向算子，而效用感知的整合则作为

离散后向算子，用于控制累积知识。第 4 节通过闭环诊断演化和验证门控近端细化来实例化这一分解。

4 方法

4.1 概述

图 1 总结了技能近端流水线。遵循第 3.3 节引入的前向-后向视角，技能近端通过前向诊断演化将初始技能 X_0 映射到中间技能 X_f ，随后通过后向近端细化生成最终技能 X^* 。前向迭代 k 使用训练批次 B_k ，而效用审计和后向验证使用固定的验证划分 V 。两个阶段共同实现诊断-近端协同演化。前向诊断根据实际执行结果演化技能的任务解决内容，而后向近端通过选择性保留和整合累积知识来演化其结构。本节其余部分遵循方法的执行顺序。我们首先介绍闭环前向更新，随后是冻结效用审计和候选构建、验证门控近端细化，以及所得过程的实现级特性。

4.2 闭环前向更新

在前向迭代 k 中，当前技能 X_k 在训练批次 B_k 上执行。诊断器利用失败轨迹、对比成功轨迹、近期历史以及上一次尝试的拒绝原因来提出编辑方向。然后，修补器从同一预迭代快照中为尝试 j 生成候选 $X_k^{(j)}$ 。该候选在同一批次上重新执行，并根据以下条件被接受：

$$\text{Gate}_{\text{fwd}}\left(\tilde{X}_k^{(j)}\right) = \mathbf{1}\left[H_{B_k}\left(\tilde{X}_k^{(j)}\right) \geq H_{B_k}\left(X_k\right)\right] \times \mathbf{1}\left[C_{B_k}\left(\tilde{X}_k^{(j)}\right) \geq C_{B_k}\left(X_k\right)\right]. \quad (5)$$

此处， $\mathbf{1}[\cdot]$ 表示指示函数，当其内部条件为真时等于 1，否则为 0。因此，前向门仅在硬准确率和平均单元准确率均满足各自接受条件时才等于 1。第一个具有严格硬准确率增益的候选会提前终止搜索。若没有候选严格提升硬准确率，则该方法最多评估三次尝试，并按硬准确率和单元准确率的字典序选择最佳尝试候选。胜出者仍须满足式 (5)；否则，该迭代保持 X_k 不变。对于被拒绝的尝试，观察到的硬/单元变化及尝试方向将提供给下一次诊断。跨迭代，紧凑的接受/拒绝摘要构成后续诊断的语义历史。式 (5) 仅约束当前训练批次 B_k 。由于不同迭代使用不同批次，这并不意味着跨迭代、验证集或测试集上的性能单调。

4.3 冻结效用审计与候选选择

前向优化后， X_f 被解析为 n 个可审计的 L2 节和 L3 引用组：

$$\mathcal{Q}(X_f) = \{q_1, q_2, \dots, q_n\}. \quad (6)$$

令 $\text{Ablate}(X_f, q_i)$ 表示 X_f 的副本，其中单元 q_i 被完全移除。对于 L2 单元，移除对应节。对于 L3 单元，移除引用文件及其所有指针。我们定义硬边际效用和单元边际效用

$$\begin{aligned} u_i^{\text{hard}} &= H_V(X_f) - H_V(\text{Ablate}(X_f, q_i)), \\ u_i^{\text{cell}} &= C_V(X_f) - C_V(\text{Ablate}(X_f, q_i)). \end{aligned} \quad (7)$$

式 (7) 中的正值表示移除该单元会降低性能；负值表示消融版本性能更好。所有效用在 Prox 之前测量一次，并在整个候选遍历过程中保持冻结。Prox 直接从原始单元效用中选择候选。候选集和处理顺序为

$$\begin{aligned} \mathcal{I}_\tau &= \{i \in \{1, \dots, n\} : u_i^{\text{cell}} < \tau\}, \tau = -0.001, \\ i < j &\iff \left(u_i^{\text{cell}}, u_i^{\text{hard}}\right) <_{\text{lex}} \left(u_j^{\text{cell}}, u_j^{\text{hard}}\right). \end{aligned} \quad (8)$$

此处， $<_{\text{lex}}$ 表示升序字典序，而非独立的数学运算符。具体而言， $(a, b) <_{\text{lex}} (c, d)$ 在以下情况成立：若 $a < c$ ，或若 $a = c$ 且 $b < d$ 。因此，候选者首先按单元效用排序，较小值优先处理，平局时按硬效用打破。审计仅决定候选者的资格与处理顺序。它评估完全消融，而收缩器可能合并、降级或移除目标。因此，每个已实现的试验必须在当前状态下重新评估。

4.4 近似：单遍验证门控收缩

令 $(i_1, \dots, i_M)$ 为按式 (8) 排序的候选者，并初始化 $X^{(0)} = X_f$ 。每个候选者至多处理一次。若先前的合并已移除其目标，则跳过该候选者。否则，收缩器在当前技能的临时副本中生成试验 T_m 。该试验必须首先满足结构有效性与严格复杂度降低：

$$\text{StructOK}(T_m) = 1, G(T_m) < G(X^{(m)}). \quad (9)$$

若式 (9) 成立，则 T_m 在固定验证划分 V 上进行评估。它必须满足硬条件、单元条件与压缩范围条件

$$\begin{aligned} H_V(T_m) &\geq H_V(X^{(m)}) - \delta_h, \\ C_V(T_m) &\geq C_V(X^{(m)}) - \delta_c, \\ 1 - \frac{G(X^{(m)})}{G(X_f)} &< \rho, \\ \delta_h &= 0, \delta_c = 0.02, \rho = 0.10. \end{aligned} \quad (10)$$

此处， $\delta_h \geq 0$ 与 $\delta_c \geq 0$ 分别为硬准确率与单元准确率允许的最大绝对单次编辑下降值，

Component	Standard PGD	Proposed method
Objective	Composite task loss f and complexity regularizer g	Two-stage task optimization and text complexity control
Forward direction	Analytical direction $-\nabla f$	Natural-language edit direction inferred from task trajectories
Forward update	$v_k = x_k - \eta \nabla f(x_k)$	Diagnose, patch, same-batch re-execute, reject feedback, and rollback
Backward step	Solve a proximal subproblem	Frozen utility audit, semantic shrinkage, structural checks, and validation gating
Locality	Distance penalty $\ x - v\ _2^2 / (2\eta)$	Local single-target edits and a soft compression cap ρ
Shrinkage strength	Controlled by $\eta\lambda$	Candidate, quality, and range controlled separately by τ , δ_h , δ_c , and ρ
Analysis	Convergence under smoothness and related assumptions	Finite termination, empirical gating, and strict complexity reduction

表 1: 标准投影梯度下降与所提方法之间的结构对齐。

而 $\rho \in [0, 1]$ 为相对于前向技能 X_f 的累积压缩阈值。我们设置 $\delta_h = 0$ ，要求记录的验证硬准确率不下降，并设置 $\delta_c = 0.02$ ，允许每次接受的编辑使平均单元准确率至多降低 0.02，或当度量归一化至 $[0, 1]$ 时为两个百分点。我们设置 $\rho = 0.10$ 。该条件在试验提交前于活动技能 $X^{(m)}$ 上评估，而非在 T_m 本身上评估。因此， ρ 并非最终输出压缩比的严格上界；它是一个软停止阈值，决定是否可进行下一次收缩尝试。只要当前累积压缩保持低于 ρ ，下一次试验即可被接受，并使最终压缩比超过 ρ 。活动状态更新为

$$X^{(m+1)} = \begin{cases} T_m, & \text{if all gates pass,} \\ X^{(m)}, & \text{otherwise,} \end{cases} \quad (11)$$

$$X^* = X^{(M)}.$$

所有编辑首先应用于试验副本。被拒绝的试验将被删除，而被接受的试验将替换活动技能。未产生任何接受删除的候选者不会终止循环；处理持续至候选序列耗尽。式 (10) 中的压缩条件在应用 T_m 之前于 $X^{(m)}$ 上评估。因此， $\rho = 0.10$ 是一个软上限：最终接受的编辑可能使总压缩超过 10%。

4.5 实现层面的性质与权衡 代理候选集是有限的，且每个候选至多被处理一次；因此，收缩过程在有限次尝试后终止。每个被接受的编辑必须严格降低文本复杂度，而被拒绝的尝试则保持当前技能不变。验证门控仅对记录的验证性能提供逐编辑的经验约束，并不蕴含测试集单调性或经典近端梯度下降收敛性。形式化陈述、累积性能界以及评估复杂度分析见附录 B。收缩强度由候选阈值、性能容差和压缩范围共同决定。

调整这些控制参数会产生具有不同文本压缩比和留出性能的技能，从而形成经验性的压缩-性能权衡。我们主要通过扫描候选阈值 τ 来研究这一关系；扫描配置、评估协议和帕累托过滤过程在第 5.4 小节中描述。重要的是， τ 在冻结效用审计下改变了候选覆盖范围，但并非复合目标中的显式正则化权重 λ 。因此， τ 扫描应被解释为阈值诱导的经验权衡曲线，而非精确的 λ 正则化路径。理想的正则化决策规则及其与 τ 的近似关系在附录 B 中讨论。

5 实验

5.1 实验设置

基准 为全面评估所提方法的有效性，我们在分布内 (IID) 和分布外 (OOD) 两种设置下开展实验。对于分布内评估，我们采用 SpreadsheetBench Verified (Ma 等, 2024)，这是 SpreadsheetBench 中经人工验证的子集，专门用于支持可靠的自动评估。对于分布外评估，我们进一步在 WikiTableQuestions (Pasupat 和 Liang, 2015) 和 HiTab (Cheng 等, 2022) 上评估所提方法，以考察所学技能在训练分布之外的泛化能力。上述数据集的详细描述见附录 C。

基线 我们将所提方法与涵盖多个代表性类别的广泛基线进行比较。首先，考虑两种基本设置：无技能 (No Skill)，即模型在没有任何辅助技能的情况下运行；以及人工技能 (Human Skill)，即提供人工精心设计的技能。需注意，人工技能同时作为技能近端优化 (SkillProx) 所优化的基础技能。其次，针对利用大语言模型 (LLM) 生成技能的方法，我们与进化技能 (EvoSkill) (Alzubi 等, 2026) 和轨迹到技能 (Trace2Skill) (Ni 等, 2026) 进行比较。第三，针对关注技能自我进化的方法，我们与技能梯度 (SkillGrad) (Wang 等, 2026b) 和技能优化 (SkillOpt) (Yang 等, 2026b) 进行比较。所有基线与所提方法均在 Qwen3.5-4B 上评估，

Model	Method	IID	OOD	
		Spreadsheet	WikiTQ	HiTab
Qwen3.5-4B	No Skill	20.3 $\pm$ 1.5	65.0 $\pm$ 3.3	61.7 $\pm$ 2.0
	Human Skill	20.3 $\pm$ 4.2	68.3 $\pm$ 1.6	63.5 $\pm$ 0.9
	EvoSkill	6.7 $\pm$ 2.1	76.8 $\pm$ 8.1	63.3 $\pm$ 2.3
	Trace2Skill	10.0 $\pm$ 0.0	66.0 $\pm$ 2.3	57.5 $\pm$ 1.7
	SkillOpt	15.3 $\pm$ 2.5	26.0 $\pm$ 6.6	16.0 $\pm$ 1.3
	SkillGrad	19.3 $\pm$ 0.5	69.7 $\pm$ 0.8	65.4 $\pm$ 3.4
	SkillProx (ours)	21.0 $\pm$ 1.4	78.5 $\pm$ 2.9	69.2 $\pm$ 2.3
Qwen3.5-27B	No Skill	44.0 $\pm$ 6.1	85.3 $\pm$ 1.8	78.5 $\pm$ 1.0
	Human Skill	38.3 $\pm$ 7.6	85.5 $\pm$ 0.0	78.7 $\pm$ 1.3
	EvoSkill	8.7 $\pm$ 3.5	86.0 $\pm$ 0.5	78.7 $\pm$ 0.3
	Trace2Skill	32.0 $\pm$ 9.6	84.2 $\pm$ 0.8	76.7 $\pm$ 1.9
	SkillOpt	51.3 $\pm$ 4.7	77.1 $\pm$ 1.9	66.8 $\pm$ 0.3
	SkillGrad	51.3 $\pm$ 0.9	85.2 $\pm$ 0.6	77.5 $\pm$ 0.4
	SkillProx (ours)	51.3 $\pm$ 1.2	86.8 $\pm$ 1.1	78.5 $\pm$ 1.1
Qwen3.6-27B	No Skill	45.3 $\pm$ 4.7	85.8 $\pm$ 0.8	78.2 $\pm$ 1.0
	Human Skill	36.7 $\pm$ 8.1	85.7 $\pm$ 1.2	78.0 $\pm$ 0.9
	EvoSkill	13.0 $\pm$ 4.4	87.7 $\pm$ 1.8	77.8 $\pm$ 2.0
	Trace2Skill	35.3 $\pm$ 3.1	85.2 $\pm$ 0.6	78.7 $\pm$ 2.3
	SkillOpt	53.3 $\pm$ 7.6	82.2 $\pm$ 0.6	79.7 $\pm$ 0.8
	SkillGrad	50.0 $\pm$ 3.3	84.8 $\pm$ 0.6	78.3 $\pm$ 0.3
	SkillProx (ours)	54.5 $\pm$ 0.5	86.2 $\pm$ 0.3	80.0 $\pm$ 1.3

表 2: 三个基准上的主要结果。R 为平均得分 (0 – 100, $\uparrow$) ; 数值为不同种子下的均值 $\pm$ 标准差。电子表格 (Spreadsheet) 为域内设置; 维基表格问答 (WikiTQ) 和分层表格 (HiTab) 为域外设置。最佳结果加粗, 次佳结果加下划线。

Qwen3.5-27B 和 Qwen3.6-27B, 以确保在不同模型规模和版本上进行全面且公平的比较。

实现细节 在训练配置方面, 我们遵循技能优化 (SkillOpt) 和技能梯度 (SkillGrad) 的设置。具体而言, 我们将每个数据集按 2:1:8 的比例划分为训练集、验证集和测试集。所有方法的最大交互轮数设为 30。对于进化技能 (EvoSkill) 和轨迹到技能 (Trace2Skill), 我们采用其官方配置以保证忠实比较。在评估协议方面, 我们以准确率作为主要度量。更多细节见附录 C。

Variant	Mean $\pm$ Std	Δ Acc.
w/o closed-loop diagnosis	53.0 $\pm$ 1.0	−1.5
w/o Prox	52.0 $\pm$ 1.0	−2.5
SkillProx (full)	54.5 $\pm$ 0.5	—

表 3: 在电子表格基准 (SpreadsheetBench) 上使用 Qwen3.6-27B 的组件消融。变体“无闭环诊断”仅保留近端梯度下降 (Prox); 变体“无近端梯度下降”保留闭环前向更新; 完整方法结合两个阶段。最佳结果加粗。

5.2 主要结果

技能近端优化 (SkillProx) 可靠地将基础技能改进为净所有骨干网络均呈现正向信号。从人类技能初始化开始优化, SkillProx 在每个骨干网络上均带来一致的提升: 它将基础技能提升了

13.0 在 Qwen3.5-27B (38.3 $\rightarrow$ 51.3) 上为正向, 且由 17.8 pp on Qwen3.6-27B (36.7 $\rightarrow$ 54.5) 实现, 同时在较小的 Qwen3.5-4B (20.3 $\rightarrow$ 21.0) 上保持轻微正向。通过将该技能视为可优化的工件而非固定输入, SkillProx 将基础初始化转化为稳定、正向的引导。

SkillProx 在域内任务上超越了 LLM 生成和自进化技能基线。在 SpreadsheetBench 上, SkillProx 在三种骨干模型 (21.0, 51.3 和 54.5) 上均取得了最佳的独立同分布准确率。相对于 LLM 生成技能的提升幅度在较弱的 4B 执行器上最为显著, 其中 SkillProx 比 EvoSkill 高出 14.3 个百分点, 比 Trace2Skill 高出 11.0 个百分点, 并且仍略胜最强的自进化基线 SkillGrad (19.3 $\rightarrow$ 21.0)。在自进化方法中, 它还表现出最低的方差 (e.g. $\pm$ 0.5 vs. $\pm$ 7.6, 针对 Qwen3.6-27B 上的 SkillOpt), 这表明其增益在不同随机种子间是稳定的, 而非有利初始化的产物。综合这些结果表明, 优化技能比仅仅生成技能能带来更可靠的行为。

SkillProx 对域外任务具有稳健的泛化能力 而不会过拟合训练分布。尽管这些技能仅在电子表格基准 (SpreadsheetBench) 上优化, 它们能够迁移到具有不同格式和输出空间的分布外 (OOD) 基准。在维基表格问答 (WikiTQ) 上, 技能代理 (SkillProx) 在通义千问 3.5-4B (Qwen3.5-4B) 上达到了最佳准确率 (78.5), 相比无技能 (No Skill) 提升了 13.5 个百分点, 相比技能梯度 (SkillGrad) 提升了 8.8 个百分点, 并且在通义千问 3.5-27B (86.8) 上也是最佳, 同时在通义千问 3.6-27B (Qwen3.6-27B) 上保持竞争力 (86.2)。在分层表格 (HiTab) 上, 它在通义千问 3.5-4B (69.2) 和

Qwen3.6-27B (80.0)。这种鲁棒性与 SkillOpt 形成鲜明对比，后者过度拟合域内风格并在分布外数据上崩溃，例如在 4B 模型上 (WikiTQ 上 26.0，HiTab 上 16.0)，甚至在 Qwen3.5-27B 上 (77.1 和 66.8)。

5.3 消融研究 我们在表 (3) 中对 SkillProx 的两个核心阶段进行了消融：闭环诊断前向更新和近端收缩后向阶段，即 Prox。由此产生两个变体。第一个移除闭环诊断，仅保留近端梯度下降。第二个保留闭环前向更新，但移除 Prox。所有组件消融均使用 Qwen3.6-27B，采用相同的训练配置和固定的训练集种子。我们采用与主实验相同的评估协议和度量，报告域内 SpreadsheetBench 设置上的结果。表 3 显示，移除任一阶段都会降低保留集准确率，相对于完整方法。移除闭环诊断将准确率从 54.5 降低至 53.0 (−1.5 个百分点)，而移除 Prox 则进一步降低至 52.0 (−2.5 个百分点)。移除 Prox 带来的更大降幅表明，仅靠任务驱动的前向编辑会积累冗余或过度实例特定的内容，需要近端收缩阶段加以控制；移除闭环诊断带来的较小但一致的降幅表明，Prox 仍需要一个优化良好的前向技能来进行收缩，而非原始基础技能。完整方法还获得了最低的方差 (± 0.5 vs. ± 1.0)，表明这两个阶段互补地产生稳定增益，而非有利初始化的产物。

5.4 详细分析

准确率 - 压缩 近端阈值分析 τ 。在图 (2) 中，我们在不重新训练的情况下，将 Prox 离线应用于 Qwen3.6-27B 产生的三个闭环技能。我们评估无 Prox 锚点 ($\tau = -\infty$)、五个负阈值和六个正阈值。无 Prox 设置实现了 OJ 困难准确率 50.3%。在 $\tau = -0.001$ 处，准确率达到最大值 52.3%，压缩率为 25.7%。在 $\tau = 0.005$ 处，压缩率增加到 41.5%，而准确率保持在 52.0%。即使在 $\tau = 0.050$, 74.9% 的技能被移除的情况下，仍保留 51.0% 的准确率。当压缩超过约 80% 时，性能开始下降。多个 Prox 设置同时实现了比无 Prox 更高的压缩率和更高的准确率，表明适度的收缩也可能减少重复规则和上下文干扰。

模型规模与最终技能长度。 我们比较了 Qwen3.5-4B 和 Qwen3.5-27B 在闭环训练和近端收缩后产生的最终 G3D 技能。平均而言，最终的 4B 技能比 27B 技能长约 45%。有趣的是，主要技能文件收敛到几乎相同的长度，约为 15k 字符。差异几乎完全来自参考文件：4B 模型产生的参考内容大约是 27B 模型的两倍。这表明模型大小影响所需补充指导的数量，而不是核心技能结构的大小。综合来看，这些结果表明较大的模型自然会产生更具选择性和更紧凑的技能，

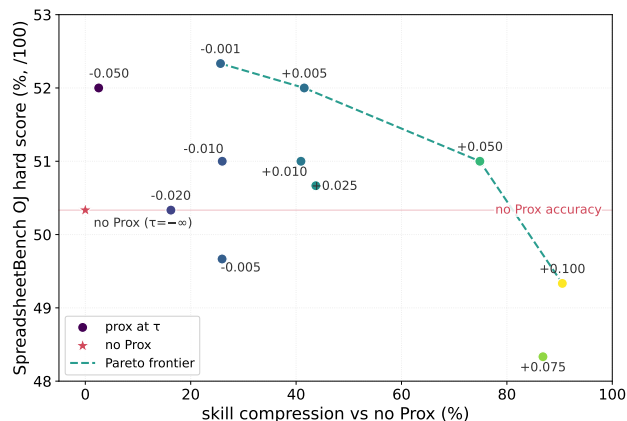


图 2: Qwen3.6-27B 在不同近端阈值下的准确率-压缩帕累托前沿 τ 。

Metric	Qwen3.5-4B	Qwen3.5-27B
Total length	40.4k $\pm$ 8.6k	27.9k $\pm$ 12.9k
Main file	14.8k $\pm$ 3.3k	15.4k $\pm$ 3.5k
References	25.6k $\pm$ 9.1k	12.6k $\pm$ 11.1k
Prox compression	29.4%	19.0%

表 4: 不同模型大小下的最终技能长度。结果在三次 4B 运行和 27B 运行上取平均。

而闭环验证和近端收缩为较小的模型提供了一种外部机制，用于拒绝不必要的更新。与这一解释一致，在 27B 运行中，G3D 长度与独立同分布困难准确率呈负相关，表明较长的技能并不一定更有效。更多结果见附录 D。

6 结论

本文提出了 SkillProx，一个受近端梯度启发的自进化智能体技能框架。SkillProx 将闭环诊断进化（通过执行反馈验证候选更新）与效用感知的近端细化（重新评估和巩固累积知识）相结合。在 SpreadsheetBench Verified、WikiTableQuestions 和 HiTab 上，SkillProx 在分布内和分布外设置下均提升了多个骨干大语言模型的技能性能。实证结果表明，通过前向-后向优化联合改进诊断更新和累积技能知识是一条实用的路径。

参考文献

- [illegible]

- C.; Zhao, F.; Zhao, J.; Zhao, S.; Zhao, X.; Zhao, Y.; Zhao, Z.; Zheng, H.; Zheng, R.; Zheng, S.; Zheng, T.; Zhong, J.; Zhong, L.; Zhong, W.; Zhou, M.; Zhou, R.; Zhou, X.; Zhou, Z.; Zhu, J.; Zhu, L.; Zhu, X.; Zhu, Y.; Zhu, Z.; Zhuang, J.; Zhuang, W.; Zou, Y.; and Zu, X. 2026a. Kimi K2.5: Visual Agentic Intelligence. *arXiv:2602.02276*.
- Team, T. D.; Li, B.; Zhang, B.; Zhang, D.; Huang, F.; Li, G.; Chen, G.; Yin, H.; Wu, J.; Zhou, J.; Li, K.; Su, L.; Ou, L.; Zhang, L.; Xie, P.; Ye, R.; Yin, W.; Yu, X.; Wang, X.; Wu, X.; Chen, X.; Zhao, Y.; Zhang, Z.; Tao, Z.; Zhang, Z.; Qiao, Z.; Wang, C.; Yu, D.; Fu, G.; Shen, H.; Yang, J.; Lin, J.; Zhang, J.; Zeng, K.; Yang, L.; Yin, H.; Song, M.; Yan, M.; Liao, M.; Xia, P.; Xiao, Q.; Min, R.; Ding, R.; Fang, R.; Chen, S.; Huang, S.; Wang, S.; Cai, S.; Shen, W.; Wang, X.; Guan, X.; Geng, X.; Shi, Y.; Wu, Y.; Chen, Z.; Li, Z.; and Jiang, Y. 2026b. Tongyi DeepResearch Technical Report. *arXiv:2510.24701*.
- Wang, H.; Lan, Y.; Cao, B.; Lin, L.; and Chen, J. 2026a. SkillGrad: Optimizing Agent Skills Like Gradient Descent.
- Wang, H.; Lan, Y.; Cao, B.; Lin, L.; and Chen, J. 2026b. SkillGrad: Optimizing Agent Skills Like Gradient Descent. *arXiv:2605.27760*.
- Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; Zheng, Z.; Xie, C.; and Yao, H. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. *arXiv:2602.08234*.
- Xu, R.; and Yan, Y. 2026a. Agent skills for large language models: Architecture, acquisition, security, and the path forward.
- Xu, R.; and Yan, Y. 2026b. Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward. *arXiv:2602.12430*.
- Yang, C.; Wu, X.; Liu, H.; Lin, X.; Xu, C.; Jiang, X.; Sun, Y.; Zhang, W.; Shi, Z.; Xu, Y.; et al. 2026a. A Survey of Agent Skills: Toward Procedural Infrastructure for LLM Agents.
- Yang, Y.; Gong, Z.; Huang, W.; Yang, Q.; Zhou, Z.; Huang, Z.; Li, Y.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, Y.; Chen, D.; Yang, X.; and Luo, C. 2026b. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. *arXiv:2605.23904*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
- Yu, J.; Zhu, J.; Lin, B.; Cui, Q.; Ding, Z.; and Li, X. 2026. Skill is Not One-Size-Fits-All: Model-Aware Skill Alignment for LLM Agents. *arXiv:2605.30723*.
- Yuan, S.; Song, K.; Chen, J.; Tan, X.; Shen, Y.; Kan, R.; Li, D.; and Yang, D. 2024. EASYTOOL: Enhancing LLM-based Agents with Concise Tool Instruction. *arXiv:2401.06201*.
- Yuksekgonul, M.; Bianchi, F.; Boen, J.; Liu, S.; Huang, Z.; Guestrin, C.; and Zou, J. 2024. TextGrad: Automatic "Differentiation" via Text. *arXiv:2406.07496*.
- Zhang, H.; Fan, S.; Zou, H. P.; Chen, Y.; Wang, Z.; Zhou, J.; Li, C.; Huang, W.-C.; Yao, Y.; Zheng, K.; Liu, X.; Li, X.; and Yu, P. S. 2026a. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. *arXiv:2604.01687*.
- Zhang, Q.; Feng, Z.; Shi, X.; Hu, X.; Liu, C.; Xie, P.; Wang, X.; Ye, J.; Hooi, B.; Wang, H.; and Zhao, J. 2026b. SkillComposer: Learning to Evolve Agent Skills for Specification and Generalization. *arXiv:2606.06079*.
- Zhou, C.; Chai, H.; Chen, W.; Guo, Z.; Shan, R.; Song, Y.; Xu, T.; Yang, Y.; Yu, A.; Zhang, W.; Zheng, C.; Zhu, J.; Zheng, Z.; Zhang, Z.; Lou, X.; Zhang, C.; Fu, Z.; Wang, J.; Liu, W.; Lin, J.; and Zhang, W. 2026a. Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering. *arXiv:2604.08224*.
- Zhou, Y.; Cao, C.; Yang, J.; Wu, L.; He, C.; Han, S.; and Guo, Y. 2026b. Lras: Advanced legal reasoning with agentic search. *arXiv preprint arXiv:2601.07296*.

附录

动机案例研究

A.1 案例研究：消除负效用冗余

本文方法针对迭代技能演化中的两种不同失效模式。前向阶段需判断所提议的更新是否真正改善行为，而后向阶段则需移除反复累积后残留的冗余或有害内容。这两个问题需要不同形式的反馈。

A.1.0.0.1 前向动机：结果盲目的技能更新 标准的前向技能演化遵循执行—诊断—修补流程，但在提交更新前不会重新执行修补后的技能。因此，一个看似合理的诊断可能在缺乏改进任务结果证据的情况下被转化为永久规则。我们在一次涉及动态变化参考值的顺序扫描的 Qwen3.6-27B 运行中观察到这一失败。开环演化将失败转化为元指令“在编码前追踪具体示例”，并包含一个带有硬编码条件值 $\geq \text{reference} * 1.10$ 的任务特定模板。留出审计对该部分赋予负的估计效用：

$$u_{\text{open}}^{\text{cell}} = -0.0337, \quad u_{\text{open}}^{\text{hard}} = -0.0556. \quad (12)$$

闭环演化遇到相同的底层失败模式，但在当前批次上重新执行每个提议的更新。退化的候选方案被回滚，其观察到的硬/单元变化返回给下一次诊断尝试。由此产生的技能反而学习到一条可操作的决策规则：在每个合格事件之后，在继续扫描之前将参考更新为当前值。该公式具有正的估计效用：

$$u_{\text{closed}}^{\text{cell}} = +0.0495, \quad u_{\text{closed}}^{\text{hard}} = +0.0474. \quad (13)$$

该对比催生了一种结果门控的前向过程。仅凭语义合理性无法区分可复用规则与过度特化的模板；同批次重新执行则为接受、拒绝或修订更新提供了可度量的信号。

A.1.0.0.2 反向动机：残差冗余与负效用 结果门控并未消除事后精炼的必要性。它仅在产生更新的批次上评估该更新，因此无法可靠地检测出在局部无害但在更广泛验证分布上冗余或有害的内容。事实上，同一次运行中的闭环技能仍包含两个负效用单元和十四个零效用单元。值得注意的是，主要的“带动态参考的顺序扫描”部分是有用的，而其支撑参考文件却被赋予负效用。因此，有用内容与有害内容可能共存于同一知识组内。前向演化还会在迭代过程中累积重复的表述。在开环技能中，“在编码前追踪一个具体示例”这一指令出现在四个部分中，单元效用从 $+0.1038$ 到 -0.0337 不等。效用最低的出现位置包含上述硬编码的任务特定模板。一次冻结效用审计在该运行中识别出五个候选单元，但只有一个候选单元在针对活跃技能的重新评估中存活下来。被接受的编辑将可迁移的状态追踪原则整合到一个正效用解释部分中，同时删除了重复的模板。它将完整技能从 29,129 个字符缩减到 28,219 个字符，对应于

$$\rho = \frac{29,129 - 28,219}{29,129} = 3.12\%. \quad (14)$$

该编辑将验证硬准确率保持在 94.74%，并将验证单元准确率从 96.05% 提高到 99.73%。这促使了一个验证门控的反向近端阶段：冻结审计识别并排序潜在冗余单元，而顺序验证决定每个已实现的编辑是否可以安全提交。

A.1.0.0.3 互补角色 两个阶段处理互补的时间尺度。闭环前向演化提供在线拦截：它衡量所提议更新的即时效果，并阻止同批次回归。反向近端提供回顾性精炼：它在冻结验证集上重新评估累积技能，并移除无法从原始批次中检测到的冗余。它们各自的角色可以概括为

$$\text{闭环前向} = \text{在线更新验证}, \quad (15)$$

$$\text{Backward Prox} = \text{post-training utility refinement}. \quad (16)$$

闭环反馈改进了知识引入的方式，而近端控制着累积后保留的内容。完整案例研究、候选级验证决策和 OJ 结果见附录 D.2 和附录 D.1。

B 附加方法细节

B.1 算法性质与复杂度

候选集是有限的，且每个候选最多被处理一次，因此主循环必然终止。每个被接受的试验必须严格小于当前技能，而被拒绝的试验则保持当前状态不变。每次被接受的编辑还保持记录到的验证硬准确率至多 $\delta_h = 0$ ，并可能使记录到的单元准确率最多降低 δ_c 。如果该方法接受 R 次收缩编辑，则记录到的验证指标满足

$$\begin{aligned} G(X^*) &\leq G(X_f), \\ G(X^*) &< G(X_f) \quad \text{if } R > 0, \\ H_V(X^*) &\geq H_V(X_f), \\ C_V(X^*) &\geq C_V(X_f) - R\delta_c. \end{aligned} \tag{17}$$

完整审计需要一次基线评估和 n 次留一法评估。Prox 最多进行 M 次收缩器调用和 M 次试验评估。如果每次评估执行所有 $|V|$ 个验证任务，则任务执行次数为

$$\mathcal{O}((n + M)|V|), \quad M \leq n. \tag{18}$$

方程 (17) 涉及记录到的验证分数。考虑到随机评估和同一验证划分的自适应重用，需要一个对所有查询联合成立的置信界；此类统计分析可置于附录中。

B.2 经验压缩 - 性能权衡

该实现没有显式的 λ ，因此我们不将其实验曲线解释为精确的 λ 正则化路径。令 $\theta = (\tau, \delta_h, \delta_c, \rho)$ 表示一个收缩配置。每个配置产生一个压缩 - 保留性能点

$$\begin{aligned} \text{Comp}(X_\theta^*) &= 1 - \frac{G(X_\theta^*)}{G(X_f)}, \\ p(\theta) &= (\text{Comp}(X_\theta^*), A_{\text{test}}(X_\theta^*)), \end{aligned} \tag{19}$$

其中 A_{test} 是在固定保留测试集或 OJ 集上的最终性能。跨技能种子和重复推理运行聚合这些点，再进行非支配过滤，得到经验帕累托近似。为了分离候选范围对压缩和性能的影响，我们使用相同的前向技能和冻结效用审计进行 τ 扫描：

$$\tau \in \{-0.05, -0.001, 0.025, 0.10\}. \tag{20}$$

随着 τ 增大，冻结效用候选集单调扩展。负阈值仅包含明显负效用的单元，而正阈值还接纳零效用和弱正效用的单元。每个阈值在同一保留 OJ 集上独立评估，未压缩的 X_f 作为零压缩锚点。数值结果、跨种子变异性、被支配点和拐点在实验中分析。阈值 τ 与正则化权重 λ 并不相同。令 $c_i > 0$ 表示候选操作 i 产生的复杂度降低。在显式正则化下，理想的单候选决策应满足

$$u_i \leq \lambda c_i. \tag{21}$$

如果候选复杂度降低近似等于一个代表性值 $\bar{c}$ ，则原始效用阈值 $u_i \leq \tau$ 允许近似

$$\lambda_{\text{eff}} \approx \frac{\tau}{\bar{c}}. \tag{22}$$

在一般情形下，原始 τ 规则对应于候选特定的隐式权重

$$\lambda_i = \frac{\tau}{c_i}. \tag{23}$$

因此， τ 是正则化压力的单调代理，而非共享的拉格朗日乘子。 $\tau < 0$ 机制执行带安全边际的负效用筛选，而 $\tau > 0$ 近似允许性能与复杂度之间的权衡。为分离 τ 对候选覆盖的影响，当前诊断扫描将硬容差和单元容差均设为 1.0，且不施加 10% 的压缩上限。因此，该扫描测量的是阈值诱导的压缩-准确率曲线，而非完整 Prox 的部署行为或显式 λ 正则化路径。

C 实验细节 本节完整记录了 Qwen3.5-4B、Qwen3.5-27B 和 Qwen3.6-27B 的实验设置，包括十次运行的 Qwen3.6-27B 消融实验以及下文两个案例研究。除非另有说明，同一骨干网络内配对的条件使用相同的数据划分、训练任务选择规则、初始技能和评估协议。

C.1 对比条件与骨干网络

四种演化技能条件构成一个二乘二的组件设计，如表 5 所示。该设计隔离了闭合前向回路与应用后向 Prox 精炼的效应。

表 5: 演化技能条件的组件设计。

Forward evolution	Without Prox	With Prox
Open-loop	G1	G2D
Closed-loop	G3f	G3D (SKILLPROX)

此外，我们评估了两个外部基线：无技能，即执行器仅接收一个最小技能存根；以及人类技能，即执行器接收一份人工编写的电子表格技能。三个骨干网络及其训练种子覆盖情况汇总于表 6。

表 6: 记录实验工件中的骨干模型与训练种子覆盖情况。

Backbone	Training seeds	Number of seeds
Qwen3.5-4B	0-2	3
Qwen3.5-27B	0-2	3
Qwen3.6-27B	0-2	3

表 7: 核心训练、精炼与评估参数。

Stage	Parameter	Setting
Generation	Temperature / thinking	0.7 / disabled
Forward evolution	Training tasks / batch size	Up to 40 / 4
Forward evolution	Planned updates / early stopping	10 / four consecutive all-correct batches
Agent runtime	Maximum turns	Executor 30; Diagnoser 15; Momentum 15; Patcher 20; Shrinker 20
Closed-loop	Attempts / hard tolerance / prior size	3 / 0 / 6 records
Closed-loop	Acceptance gate	Non-decreasing hard-correct count and mean cell accuracy
Utility audit	Validation tasks / test cases	20 / 1
Utility audit	ϵ_{cell}	0.025 for most runs
Prox	Candidate threshold	$\tau = -0.001$
Prox	Hard tolerance / cell tolerance	$\delta_h = 0 / \delta_c = 0.02$
Prox	Compression cap	$\rho = 0.10$ (soft cap)
OJ evaluation	Tasks / test cases / recalculation timeout	100 / 3 / 180 seconds
OOD evaluation	Examples / test cases	200 per benchmark / 1

C.2 执行与评分基础设施

模型请求通过轮询选择和端点故障转移分发到 vLLM 端点。电子表格输出在评分前使用 LibreOffice 重新计算，并发重新计算通过共享的办公锁进行串行化。评估任务在隔离的工作目录中运行，以便模型生成的辅助文件不会修改仓库或其他任务实例。模型推理在 NVIDIA H800 GPU 上提供服务，每个 GPU 具有 80 GB 内存。更多细节见表 7。

D 附加结果与分析

本附录呈现了关于 Qwen3.6-27B 的两个互补案例研究。第一个研究将闭环前向进化视为回归更新的在线拦截。第二个研究将后向 Prox 视为残余冗余的回顾性移除。两项研究均复用相同的训练种子，以连接两种机制；因此，它们不应被视为独立的统计证据。此外，我们提供了关于模型规模和技能长度分析的更多细节。

D.1 案例研究：闭环前向进化为何有效

D.1.0.0.1 十种子聚合比较：我们在十个 Qwen3.6-27B 训练种子上比较开环 G1 与闭环 G3f。对于每个种子，两种条件使用完全相同的训练 ID 和批次划分。评估采用规范的 OJ 协议：100 个固定测试任务，每个任务三个测试用例，仅当所有三个用例均单元完美时才视为硬成功。

表 8：十个 Qwen3.6-27B 训练种子上的规范同批次 OJ 结果。G1 和 G3f 使用相同的训练 ID 和批次划分。种子 8 在两个指标上均获得最大增益，已高亮显示。

Seed	G1 hard	G3f hard	Δ hard	G1 cell	G3f cell	Δ cell
0	47	51	+4	72.33	75.60	+3.27
1	50	51	+1	75.49	75.48	-0.01
2	49	49	0	78.40	74.91	-3.49
3	52	53	+1	77.36	80.26	+2.90
4	50	52	+2	74.56	78.65	+4.09
5	53	53	0	77.47	77.55	+0.08
6	51	52	+1	77.96	78.22	+0.26
7	51	49	-2	78.00	77.85	-0.15
8	46	51	+5	74.71	79.69	+4.98
9	54	53	-1	78.68	79.88	+1.20

在十个种子中，闭环进化在硬准确率上取得六胜两平两负，平均硬改进为 1.10 个百分点（平均单元改进为 1.31 个百分点；表 8）。跨种子标准差从 2.50 降至 1.51，最小值从 46 升至 49。最大增益出现在种子 8（46% $\rightarrow$ 51% 硬；74.71% $\rightarrow$ 79.69% 单元）。然而，种子 8 也是最弱的开环运行，其闭环结果（51%）接近 G3f 均值，而非异常强的闭环结果。因此，更合理的解释是闭环进化稳定了下尾，而非均匀提升上界。

作为描述性总结，开环硬准确率与硬改进之间的相关性为

$$\text{corr}(H_{G1}, H_{G3f} - H_{G1}) = -0.800. \quad (24)$$

该量具有信息性，但不应过度解读：由于基线同时出现在两个参数中，负号部分源于数学耦合和均值回归。

开环编辑后无任何测量。已发布的运行工件显示，开环迭代遵循执行 $\rightarrow$ 分类 $\rightarrow$ 诊断 $\rightarrow$ 动量 $\rightarrow$ 补丁。它不会再次执行更新后的技能，并且种子 8 的所有十个开环补丁均在未进行验收测试的情况下提交。闭环进化则在每次补丁后重新执行相同的四任务批次，并应用

$$\text{accept} = \mathbf{1}[H_{\text{post}} \geq H_{\text{pre}} - \tau_h \wedge S_{\text{post}} \geq S_{\text{pre}} - 10^{-9}], \quad \tau_h = 0, \quad (25)$$

其中 H 是批次中完全正确的任务数， S 是平均单元准确率。失败的编辑会回滚到迭代开始时的快照，并从相同状态重试，最多尝试三次。

在后续开环诊断中缺乏有效性信号。开环诊断既未获得拒绝上下文，也未获得先前编辑是否改变准确率的记录。其动量记录说明了修补器所做的更改，但发布的开环工件不包含后续诊断可用的编辑后结果。闭环演化增加了两个通道：

- **迭代内拒绝上下文 (F5)：** 在尝试被拒绝后，下一次诊断会收到观察到的硬分数变化和失败的编辑方向，并被指示不要重复该方向。
- **跨迭代诊断先验 (F6)：** 最近六次的接受/拒绝记录（包括硬分数变化和诊断标签）被注入到后续尝试中。

D.1.0.0.2 种子 8 的门控迹 种子 8 的闭环运行包含十次迭代和 22 次尝试编辑。八次尝试降低了同批次硬正确性，并被门控阻止（表 9）。在第 2 次迭代中，所有三次尝试均失败，因此整个迭代被回滚。相比之下，开环演化期间生成的十个补丁均未经更新后验证即被提交。

表 9：种子 8 的闭环尝试轨迹。每批包含四个任务。更新后硬正确计数低于更新前计数的尝试将被拒绝。

迭代前	尝试后计数	结果	1 2 1, 2, 2 接受；尝试 1 回退 1 并被拒绝。2 3 2, 2, 1 三个尝试均被拒绝；整个迭代被回退。3 2 3 接受；严格增益且提前终止。4 2 2, 2, 2 接受；平局通过最佳软分数解决。5 1 1, 0, 1 接受；尝试 2 回退 1 并被拒绝。6 0 0, 1 接受；尝试 2 产生严格增益。7 3 3, 2, 2 接受；尝试 2 和 3 各回退 1 并被拒绝。8 3 4 接受；严格增益。9 3 2, 4 尝试 1 被拒绝；尝试 2 产生严格增益。10 3 4 接受；严格增益。

在 10 次迭代中，闭环过程进行了 22 次尝试，阻止了 8 次硬回退，并完全回退了一次迭代。

该轨迹直接证明了补丁-重新执行-门控机制是活跃的：回退更新被在线拦截，而不是永久写入技能中。

D.1.0.0.3 相同失败模式上的效用符号翻转 两次运行都遇到一个涉及顺序扫描的失败模式，其参考值在每次合格事件后发生变化。两者都为该模式创建了一个部分，但产生的知识差异显著。

表 10：相同的失败模式在匹配的开环和闭环运行中产生性质不同的知识。

Open-loop G1	Closed-loop G3f
<i>Trace Stateful Algorithms Before Coding</i>	<i>Sequential Scan with Dynamic Reference</i>
Requires the agent to manually trace at least one example step by step before coding.	States the operational semantics directly: maintain the reference as a variable that is updated during iteration.
Encodes a task-specific 19-line template with the threshold 1.10 hard-coded:	Uses a short parameterized pattern:
<pre>count = 0 reference = data[0] for i, val in enumerate(data[1:], start=1): if val >= reference * 1.10: count += 1 if i + 1 < len(data): reference = data[i + 1]</pre>	<pre>reference = first_value count = 0 for value in remaining_values: if value >= reference * threshold: count += 1 reference = value</pre>
Its final rule is a difficult-to-falsify meta-instruction: if a small example cannot be manually verified, the logic is not ready to code.	Its final decision rule is executable: update the reference to the current value after every qualifying event; do not compare all values against one static baseline.
Negative: $u^{\text{cell}} = -0.0337, u^{\text{hard}} = -0.0556$	Positive and protected: $u^{\text{cell}} = +0.0495, u^{\text{hard}} = +0.0474$

如表 10 所示，单元效用从 -0.0337 变为 $+0.0495$ ，差值为 0.0832 。

区别在于可操作性而非篇幅。诸如“仔细追踪”之类的建议，在同批次重新执行时未必能产生可测量的行为变化。相比之下，在每个事件后更新参考会改变生成的程序，因此可以通过门控进行测试。被拒绝的编辑也为下一次尝试提供了其方向无效的证据。在这个意义上，门控选择的是可证伪的编辑：它们的行为后果可以立即被观察，有用时被接受，退化时被拒绝。这一机制与开环技能中发现的“追踪一个示例”元指令的四个重复变体一致（第 D.2.0.0.1 节）。

D.1.0.0.4 闭环演化未能解决的问题
留一法审计总结于表 11。

闭环演化并非完整的负效用过滤器。

表 11: 匹配种子 8 技能的效用构成与主文件长度。长度使用验证后的最终 SKILL.md 工件。

	Open-loop G1	Closed-loop G3f
Knowledge units	16	22
Positive	4 (25.0%)	6 (27.3%)
Zero	10	14
Negative	2 (12.5%)	2 (9.1%)
SKILL.md characters	21,191	23,748

有两个观察值得注意。首先，闭环技能更长而非更短（+2,557 字符，+12.1%）；该机制执行的是质量控制而非压缩。其次，它仍包含两个估计为负效用的单元。其中一个是在附加在上述正效用主部分上的参考文件 sequential_scan.md ($u^{\text{cell}} = -0.0395$)。因此，批次级门控可能会遗漏在其原始训练批次上无害但在更广泛验证下有害的内容。向后代理（Backward Prox）仍然具有互补性（第 D.2 节）：在该种子上，G1 在对应的规范评估下得分为 46 道 OJ 难题，G3f 得分为 51，G3f 后接 Prox-D 得分为 54。

D.2 案例研究：整合负效用冗余

D.2.0.0.1 负效用冗余：种子 8 的开环技能在四个独立章节中表达了元指令“在编码前追踪具体示例”。在 20 个任务的验证集上，使用 $\epsilon_{\text{cell}} = 0.025$ 进行留一法审计，为这些章节分配了显著不同的单元效用（表 12）。

表 12: 进化开环技能中同一元指令的四次出现。效用通过在 20 个验证任务上的留一法评估估计。

Section	u^{cell}
Ground Task Interpretation	+0.1038
Interpreting Aggregation Language	+0.0119
Verify Ambiguous Arithmetic Direction	-0.0212
Trace Stateful Algorithms Before Coding	-0.0337

效用最低的章节“编码前追踪有状态算法”与第 D.1.0.0.3 节（表 10）中识别的负效用表述相同。它不仅重复了一般性建议，还将训练任务特定的实现作为可重用模板呈现，包括硬编码条件值 $\geq$ 引用 * 1.10。因此，该章节在验证审计中被估计为有害，而非仅仅可有可无。由于审计仅使用 20 个验证任务，我们将这些分数视为候选证据，而非确定性的因果效应。

D.2.0.0.2 冻结审计与动态验证门：审计评估了 16 个知识单元，将两个分类为负效用，十个为零效用，四个为正效用。Prox 将效用为 $u^{\text{cell}} < -0.001$ 的单元视为收缩候选。每次实际移除随后在当前活跃技能下根据 $\delta_h = 0, \delta_c = 0.02$ 和软压缩上限 $\rho = 0.10$ 重新评估。任何违反验证门的尝试都会被回滚。

表 13: 候选级审计分数与实际验证决策。负审计分数决定资格，但本身并不授权删除。

Candidate	u^{cell}	u^{hard}	Decision
Trace Stateful Algorithms	-0.0337	-0.0556	Accepted
Error Handling in Aggregation	-0.0337	-0.0556	Rolled back
Verify Ambiguous Arithmetic	-0.0212	-0.0056	Rolled back
Reading Data with pandas	-0.0067	-0.0030	Rolled back
Inspect Before Operating	-0.0017	-0.0030	Rolled back

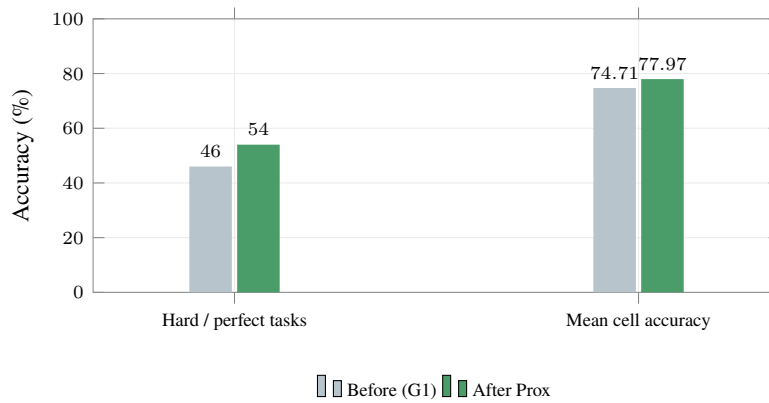


图 3: 验证门控 Prox 整合前后的 OJ 准确率。评估使用 100 个固定任务，每个任务三个测试用例；硬成功要求所有三个用例单元完美。数值未经平滑处理。

尝试了五个候选，但仅接受了一个编辑（表 13）。四个被拒绝的编辑很重要：尽管它们的冻结审计分数为负或接近零，但从当前活跃技能中删除它们会降低验证硬准确率或单元准确率。例如，移除“验证模糊算术方向”使硬准确率降低 5.85 个百分点。因此，冻结审计用于识别和排序候选；在提交编辑之前，独立的验证门仍然是必要的。

D.2.0.0.3 结构整合与 OJ 结果 被接受的编辑是一种整合而非单纯删除。Prox 移除了任务特定模板，但将可迁移原则作为一句简洁的句子保留在数据模式部分的正效用基础任务解释中。所有高正效用单元保持不变。

表 14: 被接受的 Prox 编辑所产生的结构与 OJ 变化。OJ 硬准确率使用固定的 100 任务分母。

Measure	Before	After	Change
Complete skill characters	29,129	28,219	−3.12%
SKILL.md characters	21,191	20,281	−4.29%
Second-level sections	14	13	−1
Validation hard	94.74%	94.74%	0
Validation cell	96.05%	99.73%	+3.68
OJ hard	46%	54%	+8
OJ mean cell	74.71%	77.97%	+3.26
Fail-to-pass / pass-to-fail	8 / 0		

表 14 总结了结果。完整技能从 29,129 字符缩减至 28,219 字符，

D.3 讨论

验证硬准确率保持在 94.74%，而验证单元准确率从 96.05% 提升至 99.73%。在独立的 OJ 评估中，硬准确率从 46% 提升至 54%，平均单元准确率从 74.71% 提升至 77.97%（图 3）。任务级比较包含八个失败转通过跃迁，且无通过转失败跃迁；八个跃迁中有一个是由 Prox 前条件下的推理应用程序编程接口错误引起的，不应解释为技能效应。保守地排除该任务后，仍留有七个分级失败转通过改进（表 15）。

表 15: Prox 整合后的任务级失败转通过跃迁。无任务表现出通过转失败跃迁。

Task	Type	Cell accuracy	Behavioral difference
469-9	Sheet	0.100 → 1.000	Correctly determines the debit/credit sign direction.
230-16	Sheet	0.222 → 1.000	Scans upward to find the true final row instead of trusting max_row.
38537	Cell	0.441 → 1.000	Identifies the true data range B3:B36.
35739	Cell	0.475 → 1.000	Correctly handles a cutoff time crossing midnight.
38969	Cell	0.526 → 1.000	Corrects the nested IF logic and column scan range.
51090	Cell	0.818 → 1.000	Completes multi-sheet filtered aggregation (18/22 → 22/22).
16511	Cell	0.889 → 1.000	Assigns increasing track numbers within each album.

分级改进集中在数据范围识别和任务语义上，这正是保留的检查与解释部分所承担的角色。这种对应关系是描述性的：因为两个 OJ 条件

独立世代中，它并未将接受的编辑认定为每次转变的唯一原因。这些开集结果应被解释为在温度 0.7 下独立生成时的关联改进，而非单一接受编辑的严格因果效应。本案例支持的更强因果主张是局部的：验证门控的近端策略识别出负效用表述，移除其任务特定模板，并在不降低验证门控的情况下巩固可迁移原则。两个案例研究考察了同一种子上相关累积失败的不同处理方式。闭环前向进化提供在线拦截：它测量提议更新的即时批次级效果并阻止严重回退。后向近端策略提供回顾性精炼：它在冻结验证集上重新评估累积技能，并巩固源自批次中无法检测的冗余或估计为负的内容。将近端策略-去噪应用于闭环种子 8 技能，得到开集硬准确率为 54%，与两阶段可组合的观点一致。这些发现应谨慎解读。首先，两个案例研究均复用种子 8，该种子因表现出最大闭环增益且是最弱的开环运行而被选中；其 +5 增益不得外推为平均处理效应。其次，两条技能轨迹早期即分化，因此效用符号翻转是观察性的，而非受控的 A/B 编辑。第三，开集条件是温度 0.7 下的独立生成，而非固定采样随机性的成对重掷。第四，效用审计仅使用 20 个验证任务。最后，已发布的种子 8 闭环结果未记录 pre_soft 或尝试级拒绝标签；表 9 中的尝试级严重回退标注是通过比较记录的前后正确计数重建的，而迭代级标签可用。因此，这些案例的价值在于机制层面：它们展示了在线结果门控与训练后近端策略如何应对同一累积问题的不同时间尺度。

D.4 模型规模与技能更新的补充分析 **D.4.0.0.1 最终技能长度与独立同分布准确率** 本文考察更长的最终技能是否与更强的领域内性能相关。此处，最终技能指 G3D，即经过闭环训练和 Prox-D 后获得的技能。技能长度以主 SKILL.md 文件及其参考文件中 Markdown 字符总数衡量；智能体生成的临时文件不计入。独立同分布性能采用 SpreadsheetBench OJ 硬准确率在固定的 100 个任务测试集上衡量，每个生成程序应用于全部三个测试用例，仅当三个用例全部通过时任务才计为正确。图 4 展示了九个 Qwen3.5-27B 训练种子。最终技能长度与独立同分布硬准确率呈负相关 ($r = -0.628, n = 9$)。特别地，最长的技能并未达到最高准确率，表明积累更多文本指导未必有益。拟合线仅作为描述性总结，不代表因果关系。该估计对种子 8 敏感；移除该种子后相关性降至 $r = -0.330$ 。因此，该结果应被解读为技能长度本身并非技能质量的可靠代理指标，而非统计上确凿的单调效应。

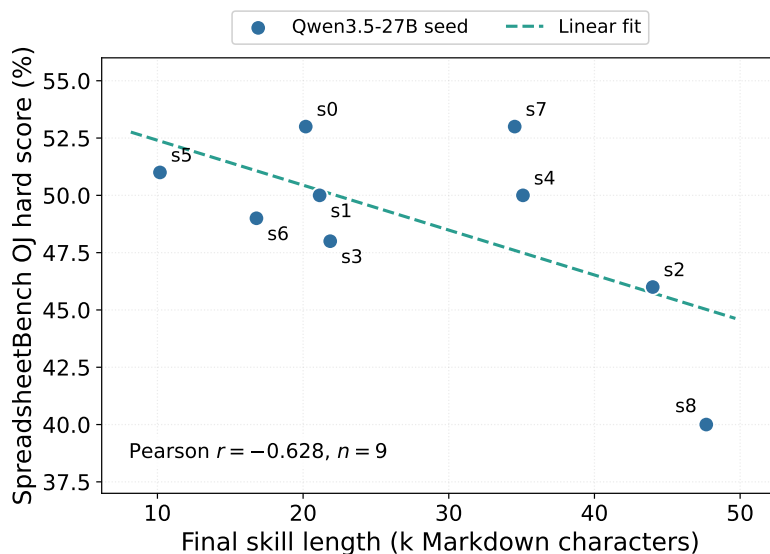


图 4：九个 Qwen3.5-27B 训练种子的最终 G3D 技能长度与 SpreadsheetBench 独立同分布 OJ 硬准确率。长度包括主技能文件及参考文件中的 Markdown 内容。虚线为普通最小二乘拟合。

D.4.0.0.2 闭环更新动态 为理解模型规模为何影响最终技能表示，本文分析内部闭环轨迹（表 16）。由于一次诊断可能触发多次写入，每次诊断的写入次数是更新强度的代理指标，而非字面概率。

表 16：闭环更新动态。数值为三个 4B 运行和九个 27B 运行的均值 $\pm$ 样本标准差。

Metric	Qwen3.5-4B	Qwen3.5-27B
Attempts / iteration	2.57 $\pm$ 0.15	2.13 $\pm$ 0.36
Diagnoses / run	102.7 $\pm$ 6.1	85.3 $\pm$ 14.6
Characters / diagnosis	1355 $\pm$ 75	1474 $\pm$ 129
Write calls / run	57.3 $\pm$ 9.9	35.6 $\pm$ 8.2
Accepted iterations	66.7% $\pm$ 15.3%	85.6% $\pm$ 10.1%
G1 writes / diagnosis	0.96	0.40
G3f writes / diagnosis	0.56	0.42
Realized Prox compression	29.4% $\pm$ 17.9%	19.0% $\pm$ 18.1%

4B 模型需要更多的内循环尝试，产生更多的诊断，并且比 27B 模型执行了显著更多的写入调用。更重要的是，对于 4B 模型，每次诊断的写入次数从开环 G1 训练下的 0.96 下降到闭环 G3f 训练下的 0.56。相比之下，27B 模型的相应值几乎保持不变（0.40 $\rightarrow$ 0.42）。门控机制还拒绝了更多来自 4B 模型的最终更新：其迭代中仅 66.7% 保留了提议的更新，而 27B 为 85.6%。这些观察表明，较大的模型已经相对有选择性地判断诊断是否应修改技能，而较小的模型在开环训练下倾向于更积极地将诊断转化为编辑。因此，补丁-重新执行-门控机制充当了一个外部更新过滤器，对较小的模型进行了不成比例的正则化。Prox-D 通过将更强的实际压缩应用于 4B 技能而非 27B 技能（29.4% 对 19.0%），强化了相同的效果。总之，闭环验证和近端收缩使较小的模型走向更具选择性和紧凑的更新机制。

E 提示与算法

本附录给出了 SkillProx 中的一些代表性提示，其他一些提示遵循 SkillGrad 的设置（Wang 等人，2026a）。最后，我们提供了 SkillProx 的整体算法。

E.1 动量提示

动量代理将每个任务的诊断转换为一个持久的模式记录和一个当前迭代的覆盖层。前者携带复发和补救历史；后者为修补器提供每个任务的一个可操作条目。

Pattern-record writer (momentum) prompt

```
1  [SYSTEM]
2  You are PatternRecordWriter.
3
4  Read:
5  1. batch_diagnoses.md;
6  2. the previous momentum_memory.md (empty at iteration 1); and
7  3. the current SKILL.md and references.
8
9  Write:
10 1. 更新后的跨迭代模式记录；以及 11 2. 与诊断顺序相同的逐任务覆盖层。12 13 模式是错误或成功的类别，而非任务实例。默认将证据合并到现有模式中。仅当所需操作或纠正措施确实不同时才进行拆分。将 remedy_log 保留为仅追加历史。17 18 模式记录条目：19 ### {模式标识} | {operation|workflow|mixed} | {描述} 20 - 锚点：{二级章节标识 | 三级文件名 | (暂无)} 21 - appeared_in: {迭代} 22 - 描述：{累积证据} 23 - latest_executor_action: {当前可操作规则} 24 - remedy_log: 25 - iterk | 诊断：{摘要} | 补丁：{操作} 26 27 覆盖层条目：28 ### [{task_id}] {信号描述} 29 - 信号：失败 | 成功 30 - 模式：{模式标识 | 新 | 无可操作信号} 31 - 锚点：{锚点} 32 - 差距：{缺失或误触发行为} 33 - proposed_change: {定向编辑方向} 34 35 如果诊断过于模糊，则使用“无可操作信号”并省略差距和 36 proposed_change. 最后以最多三个反复出现的工作流主题结束。37 38 [用户] 39 这是第 {k} 次迭代。40 41 输入：42 当前迭代诊断：{batch_diagnoses_path} 43 先前模式记录：{previous_memory_path} 44 当前指导文件：45 - {skill_md_path} 46 - {reference_paths} 47 48 输出：49 更新后的模式记录：{memory_output_path} 50 逐任务覆盖层：{overlay_output_path}
```

E.2 修补器提示

修补器对技能应用一次分层感知更新。它同时读取原始诊断和动量输出，以便动量代理的压缩不会抹去任务级证据。

Skill patcher prompt

```
1 [SYSTEM]
2 You are Patcher. Edit the existing skill in place; never redraft it wholesale.
3
4 The skill has three layers:
5 - L1 metadata routes skill selection.
6 - L2 SKILL.md is always loaded and must remain general and skimmable.
7 - L3 references are loaded conditionally and hold specific operations,
8   runtime branches, worked examples, and verification procedures.
9
10 Iterate by pattern, not by task:
11 1. Group overlay entries sharing a pattern.
12 2. Read their pattern history and raw diagnoses.
13 3. Brainstorm 2--3 candidate remedies.
14 4. Apply the simplest edit that generalizes.
15
16 Use YAGNI. Prefer extending or merging an existing section over creating a
17 new one. Never place task-specific columns, rows, filenames, or constants in
18 L2. Every L3 file must have exactly one L2 pointer and must contain runnable
19 code, a runtime branch, and a verification step with a corrective action.
20
21 After editing, read back all changed files and repair broken pointers,
22 orphaned references, duplicate sections, and workflow-checklist growth.
23
24 [USER]
25 基于以下分析演进技能。26 27 原始诊断: {batch_diagnoses_path} 28 逐尝试覆盖层:
{momentum_overlay_path} 29 跨迭代模式记录: {momentum_memory_path} 30 31 技能文件: 32
- {skill_md_path} 33 - {reference_paths} 34 35 新三级文件的参考目录: {references_directory}
```

E.3 收缩器提示

效用审计本身没有生成式提示：它在完整技能和留出技能上反复调用执行器。一旦确定性审计选定候选，收缩器恰好接收一个目标单元。

Prox shrinker prompt

```
1 [SYSTEM]
2 You are Shrinker. The forward optimizer only added content; make the skill
3 smaller and cleaner without losing what works.
4
5 The target is an L2 section or L3 reference whose measured marginal utility
6 is low or negative. Diagnose it as:
7 (a) redundant, (b) contradictory or misleading,
8 (c) over-specific or verbose, or (d) genuinely load-bearing.
9
10 Apply the lightest valid operation:
11 1. Consolidate (preferred): salvage unique generalizable content into the
12   most-overlapping retained section, then remove the target.
13 2. Demote: move excessive L2 detail into a concise L3 procedure.
14 3. Remove: delete content that is fully redundant or misleading.
```

```

15
16 Touch only the target and, when needed, one receiving section. Preserve all
17 L2--L3 pointer invariants. A downstream validation gate will reject the edit
18 if accuracy drops. The resulting skill must be strictly smaller.
19
20 [USER]
21 Shrink the skill by processing exactly one target unit.
22
23 Target unit (flagged low/negative marginal value):
24 {target_L2_section_or_L3_reference}
25
26 Skill files:
27 - {skill_md_path} 28 - {reference_paths} 29 30 从目标中挽救真正有用、可泛化的内容到重叠最多的保留章节。删
除冗余、矛盾或冗长的剩余部分。保持所有其他章节不变。

```

E.4 端到端技能代理

算法 1 涵盖数据构建、开环或闭环前向演化、留一法效用估计以及验证门控的近端反向传播。闭环分支描述了由已发布运行工件所代表的多尝试行为：被拒绝的编辑会回滚并最多重试三次，同时将反馈注入下一次诊断。

Algorithm 1: SkillProx: closed-loop forward evolution and Prox consolidation

Require: Initial skill S_0 ; model M ; evolution pool $\mathcal{D}_e$; validation set $\mathcal{V}$; mode $m \in \{\text{open}, \text{closed}\}$; batch size b ; iterations K ; attempts A ; utility threshold τ ; compression cap ρ ; validation tolerances (δ_h, δ_c)

Ensure: Forward skill S_{fwd} and consolidated skill S_{prox}

- 1: **Phase I: Construct failure-driven update batches**
- 2: $R_0 \leftarrow \text{ExecuteAndAssess}(M, S_0, \mathcal{D}_e)$
- 3: $\mathcal{F} \leftarrow \{x \in \mathcal{D}_e : \text{Cell}(R_0[x]) < 1\}$
- 4: $\mathcal{T} \leftarrow \text{SampleUpTo40}(\mathcal{F}, \text{training seed})$
- 5: $(B_1, \dots, B_K) \leftarrow \text{Chunk}(\mathcal{T}, b)$
- 6: $S \leftarrow \text{Copy}(S_0)$; $P \leftarrow \emptyset$; $Q \leftarrow \emptyset$; $c_{\text{all}} \leftarrow 0$
- 7: **Phase II: Forward skill evolution**
- 8: **for** $k = 1, \dots, K$ **do**
- 9: $S^{(0)} \leftarrow \text{Copy}(S)$ {iteration snapshot}
- 10: $R_{\text{pre}} \leftarrow \text{ExecuteAndAssess}(M, S, B_k)$
- 11: $h_{\text{pre}} \leftarrow \text{Hard}(R_{\text{pre}})$; $s_{\text{pre}} \leftarrow \text{Cell}(R_{\text{pre}})$
- 12: $(\mathcal{F}_k, \mathcal{C}_k) \leftarrow \text{Classify}(R_{\text{pre}})$
- 13: **if** $\mathcal{F}_k = \emptyset$ **then**
- 14: $c_{\text{all}} \leftarrow c_{\text{all}} + 1$
- 15: **if** $c_{\text{all}} = 4$ **then**
- 16: **break**
- 17: **end if**
- 18: **else**
- 19: $c_{\text{all}} \leftarrow 0$
- 20: **end if**
- 21: $a_{\text{max}} \leftarrow A$ if $m = \text{closed}$, else 1
- 22: $C \leftarrow \emptyset$ {gate-eligible candidate skills}
- 23: $r \leftarrow \emptyset$ {within-iteration rejection context}
- 24: **for** $a = 1, \dots, a_{\text{max}}$ **do**
- 25: **if** $a > 1$ **then**
- 26: $S \leftarrow \text{Restore}(S^{(0)})$
- 27: **end if**
- 28: $q \leftarrow \text{FormatPrior}(Q, \text{last} = 6)$
- 29: $D_k \leftarrow \text{Diagnose}(M, \mathcal{F}_k, \mathcal{C}_k, R_0, R_{\text{pre}}, q, r)$
- 30: **if** $k = 1$ **then**
- 31: $O_k \leftarrow \emptyset$ {bootstrap has no prior momentum}
- 32: **else**
- 33: $(P, O_k) \leftarrow \text{Momentum}(M, D_k, P, S)$


```

34:   end if
35:    $\tilde{S} \leftarrow \text{Patch}(M, S, D_k, P, O_k)$ 
36:   if  $m = \text{open}$  then
37:      $S \leftarrow \tilde{S}$  {open-loop patch is committed}
38:     break
39:   else
40:      $R_{\text{post}} \leftarrow \text{ExecuteAndAssess}(M, \tilde{S}, B_k)$ 
41:      $h_{\text{post}} \leftarrow \text{Hard}(R_{\text{post}}); s_{\text{post}} \leftarrow \text{Cell}(R_{\text{post}})$ 
42:      $g \leftarrow [h_{\text{post}} \geq h_{\text{pre}}] \wedge [s_{\text{post}} \geq s_{\text{pre}} - 10^{-9}]$ 
43:     if  $g$  then
44:        $C \leftarrow C \cup \{(h_{\text{post}}, s_{\text{post}}, \text{Copy}(\tilde{S}), D_k)\}$ 
45:       if  $h_{\text{post}} > h_{\text{pre}}$  then
46:         break {strict hard improvement}
47:       end if
48:     else
49:        $S \leftarrow \text{Restore}(S^{(0)})$ 
50:        $r \leftarrow \text{RejectContext}(h_{\text{pre}}, s_{\text{pre}}, h_{\text{post}}, s_{\text{post}}, \tilde{S})$ 
51:     end if
52:   end if
53: end for
54: if  $m = \text{closed}$  then
55:   if  $C \neq \emptyset$  then
56:      $(h^*, s^*, S, D_k) \leftarrow \arg \max_{(h, s, \cdot, \cdot) \in C} (h, s)$ 
57:      $Q \leftarrow Q \cup \{(k, \text{accepted}, h_{\text{pre}}, h^*)\}$ 
58:   else
59:      $S \leftarrow \text{Restore}(S^{(0)})$ 
60:      $Q \leftarrow Q \cup \{(k, \text{rejected}, h_{\text{pre}}, h_{\text{pre}})\}$ 
61:   end if
62: end if
63: if  $k = 1$  then
64:    $(P, \_) \leftarrow \text{Momentum}(M, D_k, \emptyset, S)$  {bootstrap persistent pattern record}
65: end if
66: end for
67:  $S_{\text{fwd}} \leftarrow \text{Copy}(S)$ 
68: Phase III: Leave-one-out utility audit
69:  $U \leftarrow \text{ParseUnits}(S_{\text{fwd}})$ 
70:  $v_0 \leftarrow \text{Evaluate}(M, S_{\text{fwd}}, \mathcal{V})$ 
71: for all  $u_i \in U$  do
72:    $S_{-i} \leftarrow \text{MaterializeWithout}(S_{\text{fwd}}, u_i)$ 
73:    $v_{-i} \leftarrow \text{Evaluate}(M, S_{-i}, \mathcal{V})$ 
74:    $u_i^{\text{cell}} \leftarrow v_0^{\text{cell}} - v_{-i}^{\text{cell}}$ 
75:    $u_i^{\text{hard}} \leftarrow v_0^{\text{hard}} - v_{-i}^{\text{hard}}$ 
76: end for
77:  $L \leftarrow \text{SortAscending}(\{u_i : u_i^{\text{cell}} < \tau\}, (u_i^{\text{cell}}, u_i^{\text{hard}}))$ 
78: Phase IV: Validation-gated Prox consolidation
79:  $S_{\text{prox}} \leftarrow \text{Copy}(S_{\text{fwd}})$ 
80:  $v \leftarrow \text{Evaluate}(M, S_{\text{prox}}, \mathcal{V}); n_0 \leftarrow |S_{\text{prox}}|$ 
81: for all  $u_i \in L$  do
82:   if  $(n_0 - |S_{\text{prox}}|)/n_0 \geq \rho$  then
83:     break
84:   end if
85:    $T \leftarrow \text{Copy}(S_{\text{prox}})$ 
86:    $T \leftarrow \text{Shrink}(M, T, u_i)$  {consolidate, demote, or remove}
87:    $T \leftarrow \text{GarbageCollectOrphanReferences}(T)$ 
88:   if  $\neg \text{StructurallyValid}(T) \vee |T| \geq |S_{\text{prox}}|$  then
89:     continue
90:   end if
91:    $\tilde{v} \leftarrow \text{Evaluate}(M, T, \mathcal{V})$ 
92:   if  $\tilde{v}^{\text{hard}} \geq v^{\text{hard}} - \delta_h$  and  $\tilde{v}^{\text{cell}} \geq v^{\text{cell}} - \delta_c$  then
93:      $S_{\text{prox}} \leftarrow T; v \leftarrow \tilde{v}$  {otherwise the trial is discarded}

```

```
94:   end if  
95: end for  
96: return  $S_{\text{fwd}}, S_{\text{prox}}$ 
```